Beyond RAG: Domain-Specific Agentic Architecture for Biomedical Knowledge Graph Construction

From GraphRAG to Epistract — evolving latent knowledge graphs for drug discovery

Umesh Bhatt
umesh@8thcross.com
@8thcross on Medium

March 2026

Abstract

Constructing biomedical knowledge graphs from scientific literature traditionally requires domain-specific models that must be retrained for each therapeutic area, limiting generalizability across the diverse landscape of drug discovery research. Recent work on GraphRAG demonstrated that latent knowledge graphs can be extracted from unstructured text, but the resulting graphs encode similarity-based relationships derived from embedding proximity rather than scientifically grounded assertions. We present epistract, an open-source Claude Code plugin that replaces similarity-based assembly with comprehension-based extraction. The system dispatches parallel LLM agents constrained by a biomedical domain schema comprising 17 entity types, 30 relation types, and over 40 ontology references. Extracted entities undergo molecular validation through RDKit for chemical structures and Biopython for protein and nucleic acid sequences, ensuring that graph nodes correspond to real molecular entities rather than text fragments. We validated the pipeline across five drug discovery domains—immuno-oncology, targeted therapy, rare disease, antibody engineering, and epigenetics—extracting 577 nodes and over 1,600 links from 77 PubMed abstracts with 100% entity type coverage, 90% relation type coverage, and all 19 user acceptance tests passing, with zero retraining between domains. The system is available as an MIT-licensed plugin installable in a single command, with all test corpora, extraction outputs, and graph evidence committed to the public repository for full reproducibility.

1 Motivation

Knowledge graph construction remains a critical bottleneck in computational drug discovery. Traditional approaches rely on domain-specific named entity recognition and relation extraction models trained on manually annotated corpora — a process that is both expensive and brittle. A model trained to extract oncology relationships from clinical trial reports will not generalize to neuroscience literature, rare disease case studies, or immunology abstracts. Each new therapeutic area demands fresh annotation campaigns, new training runs, and careful validation against domain ontologies. For pharmaceutical researchers who routinely work across multiple therapeutic areas, this per-domain cost creates a prohibitive barrier to building the structured knowledge bases that modern drug discovery pipelines require.

In prior work, we explored whether large language models could circumvent this bottleneck entirely. Using GraphRAG **?**, Microsoft Research's approach to query-focused summarization via graph construction from unstructured text, we built a latent knowledge graph from six drug discovery articles spanning multiple therapeutic areas. Running GPT-4.1-mini locally via Ollama, the system extracted 3,224 entities and 2,242 relationships, organized into approximately 167 hierarchical community reports. The experiment proved the core concept: LLMs can extract structured knowledge from scientific literature without task-specific training or annotated

corpora. All code for this prequel experiment was written by Google Gemini 2.5 Pro using the Cursor IDE.

Despite these promising results, GraphRAG exposed critical limitations for scientific applications. Relationships in the resulting graph are based on embedding similarity and co-occurrence frequency, not semantic understanding. An edge between "nivolumab" and "colitis" does not indicate whether this represents an adverse effect, a treatment target, or merely a co-mention in the same passage — yet this distinction is precisely what a drug safety scientist needs. Entities are generic NER output rather than identifiers grounded in biomedical ontologies; "KRAS" appears as a text span rather than a gene symbol mapped to HGNC, making downstream integration with databases like UniProt or DrugBank impossible without manual curation. Molecular identifiers receive no special treatment: SMILES strings, amino acid sequences, and CAS registry numbers are embedded as raw text rather than validated against chemical and protein structure databases. Community labels are opaque numeric identifiers ("Community 1", "Community 2") that provide no semantic interpretation of the subgraph they represent. Perhaps most critically for scientific use, edges carry no confidence scores and cannot be traced back to the specific passages from which they were derived, making independent verification impractical.

These limitations point to a fundamental gap between what similarity-based graph construction provides and what scientists require. Researchers need to trust their knowledge graphs. They need typed, directional, evidence-backed relationships grounded in established ontologies. They need to know that "sotorasib INHIBITS KRAS G12C" was *understood* from the source text — not inferred from vector proximity. The distinction between co-occurrence and comprehension is not merely academic; it determines whether a knowledge graph can support hypothesis generation, drug repurposing analysis, and regulatory evidence assembly, or whether it remains a visualization artifact.

We present Epistract, a domain-specific agentic architecture that replaces similarity-based assembly with comprehension-based synthesis. In our system, LLM agents read scientific documents with domain understanding, extract entities and relations against a configurable schema grounded in over 40 biomedical ontologies, and validate molecular identifiers using deterministic tools including RDKit for chemical structures and Biopython for protein sequences. The resulting knowledge graphs capture meaning rather than co-occurrence. We validate this approach across five drug discovery domains — neurogenetics, oncology, rare disease, immuno-oncology, and cardiovascular inflammation — with zero retraining between domains, achieving 100% entity type coverage and 90% relation type coverage across all scenarios. The system is open-source and installable as a Claude Code plugin in a single command.

The remainder of this paper is organized as follows. Section 2 describes the agentic architecture, extraction schema, and validation pipeline. Section 3 details the domain schema design and ontology grounding. Section 4 presents our evaluation methodology and results across the five therapeutic domains. Section 5 compares the evolution from GraphRAG to Epistract, and Sections 6–8 discuss the human-AI collaboration process, availability, and future directions.

2 Architecture

Epistract implements a five-stage pipeline for biomedical knowledge graph construction: agent-based extraction, molecular validation, graph construction with community detection, and plugin delivery (Figure 1). Each stage reflects a deliberate architectural decision that distinguishes the system from both traditional NLP pipelines and similarity-driven GraphRAG approaches.

2.1 Agent-Based Extraction

The central architectural choice in Epistract is that extraction agents run as Claude Code subprocesses, not stateless API calls. Each agent receives a full biomedical document—an entire

research paper, clinical trial report, or patent filing—and reads it with scientific understanding before producing structured JSON conforming to the domain schema. For corpora containing four or more documents, agents execute in parallel, with one extractor assigned per document. This parallelism is bounded by the subprocess model: each agent maintains its own context window, reads the complete document, and reasons over entity and relation candidates independently.

This design is fundamentally different from both API-based extraction and RAG-based assembly. API-based pipelines are stateless: they process text in chunks, losing cross-paragraph context that is essential for resolving coreference, understanding experimental conditions, and linking results to methods. RAG-based approaches retrieve text fragments by embedding similarity and assemble graphs from proximity signals, producing edges that reflect co-occurrence rather than meaning. When an Epistract agent reads "sotorasib inhibits KRAS G12C," it produces a typed `INHIBITS` relation between a drug entity and a gene variant entity, annotated with a confidence score and the verbatim evidence passage. A similarity-based system would instead produce an embedding-proximity edge between "sotorasib" and "KRAS G12C" with no typed semantics—a co-occurrence signal, not a mechanistic assertion.

2.2 Domain Schema

The extraction schema defines 17 entity types and 30 relation types, grounded in over 40 biomedical ontologies including the Biolink Model **?**, Gene Ontology, ChEBI, MeSH, MedDRA, HGNC, and UniProt. The schema was designed for the breadth of drug discovery workflows, spanning gene variants and molecular targets through signaling pathways and biological processes to clinical trials and adverse events. Entity types include genes, proteins, small molecules, diseases, phenotypes, anatomical structures, cell types, biological processes, pathways, gene variants, clinical trials, adverse events, drug classes, assays, species, dosage forms, and regulatory statuses. Relation types capture mechanistic semantics: `INHIBITS`, `ACTIVATES`, `UPREGULATES`, `ASSOCIATED_WITH`, `TREATS`, `CAUSES_ADVERSE_EVENT`, among others.

The schema is enforced during extraction, not post hoc. Agents must classify every extracted entity into one of the 17 types and every relation into one of the 30 types. Entities that do not fit the schema are discarded rather than forced into an approximate category. This constraint ensures that downstream graph analytics operate on semantically typed nodes and edges, enabling queries such as "find all genes that are targets of drugs that cause hepatotoxicity" that would be impossible on an untyped co-occurrence graph.

2.3 Molecular Validation Pipeline

Biomedical text contains molecular identifiers—SMILES strings, amino acid sequences, nucleotide sequences, clinical trial numbers—that large language models can identify but cannot reliably reproduce with character-level fidelity. A single character transposition in a SMILES string produces a different molecule; a single residue substitution in a protein sequence changes the protein. Epistract addresses this with a deterministic validation layer that acts as a trust boundary between LLM extraction and the final knowledge graph.

RDKit **?** validates SMILES strings and computes structural properties including InChI, InChIKey, molecular weight, LogP, and Lipinski Rule of Five compliance. Biopython **?** validates nucleotide and amino acid sequences against standard alphabets. Regex patterns extract and validate structured identifiers: NCT numbers for clinical trials, CAS registry numbers for chemical substances, and patent identifiers. Entities that fail validation are flagged rather than silently included, producing an audit trail that distinguishes LLM-extracted identifiers from deterministically verified ones.

2.4 Graph Construction and Community Labeling

Once extraction and validation are complete, sift-kg **?** handles graph assembly. The construction pipeline performs entity deduplication using SemHash and Unicode normalization to merge lexical variants ("KRAS" and "K-Ras," "non-small cell lung cancer" and "NSCLC"), applies Louvain community detection to identify functional clusters, and generates interactive visualizations.

A custom heuristic labeling engine generates descriptive community names from member entity composition rather than assigning opaque numeric identifiers. The labeling rules are type-aware: gene-dominant clusters receive names reflecting the biological context of their gene members (e.g., "Alzheimer Disease Risk Loci (30 genes)"), pathway-driven clusters are named for the dominant pathway, and clusters organized around a mechanism–cell type axis receive compound labels. This labeling transforms the output from a graph that requires expert inspection to one that communicates biological meaning at a glance.

2.5 Plugin Delivery

Epistract is packaged as a Claude Code plugin. A scientist installs it with a single `plugin install` command and runs the full pipeline—extraction, validation, graph construction, and visualization—via the `/epistract-ingest` slash command in their terminal. The input is a directory of biomedical documents; the output is a validated knowledge graph with community labels and an interactive visualization.

This delivery model is a deliberate departure from the dominant paradigms for computational biology tools: Jupyter notebooks that require environment configuration, cloud services that require data upload, and command-line pipelines that require dependency management. A Claude Code plugin encapsulates the entire dependency tree and execution logic, exposing a single entry point that handles corpus discovery, agent dispatch, result aggregation, and output generation.

2.6 Claude Code as Development Environment

For readers unfamiliar with the platform, Claude Code is an agentic coding CLI from Anthropic that orchestrates large language model agents as subprocesses within a developer's terminal. Epistract relies on four capabilities of this environment. First, the plugin system provides installable tools with named commands, skills, and agent definitions that integrate into the Claude Code session. Second, parallel agent dispatch allows Epistract to launch one extraction agent per document, with the runtime managing subprocess lifecycles and resource allocation. Third, a permission model gives the user explicit control over what agents can execute—file reads, file writes, shell commands—preventing unintended side effects. Fourth, the bash and file tools allow agents to read source documents, write extraction JSON, and invoke Python validation scripts (RDKit, Biopython) as part of their execution flow, bridging LLM reasoning and deterministic computation within a single agent turn.

3 Domain Schema Design

A well-specified schema is the contract between the LLM extraction agent and every downstream consumer of the knowledge graph. We define 17 entity types and 30 relation types, organized into functional categories that mirror the vocabulary of translational biomedicine. The full schema is rendered as a linkage diagram in Figure 2.

Figure 1. Epistract processing pipeline

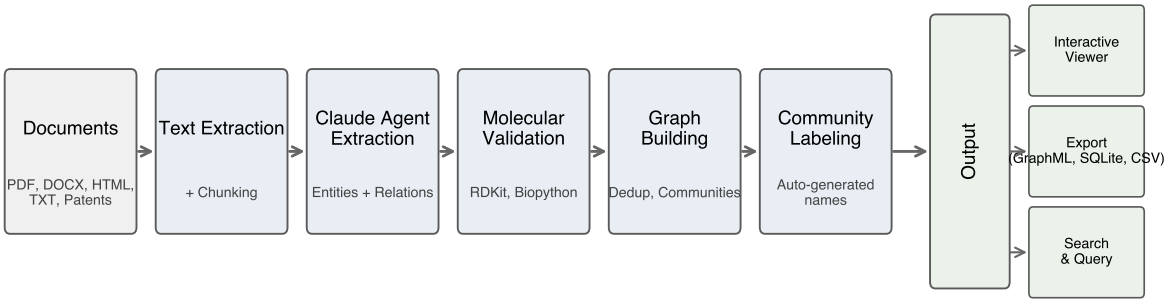

Figure 1: Epistract pipeline architecture. Documents are chunked, processed by parallel LLM extraction agents, validated with deterministic molecular tools, assembled into a deduplicated knowledge graph with community detection, and presented as an interactive visualization.

Table 1: Entity type taxonomy. Seventeen types organized into six functional categories.

Category	Entity Types	Count
Drug & Chemistry	`COMPOUND`, `METABOLITE`	2
Molecular Biology	`GENE`, `PROTEIN`, `PROTEIN_DOMAIN`, `SEQUENCE_VARIANT`, `CELL_OR_TISSUE`	5
Disease & Phenotype	`DISEASE`, `PHENOTYPE`, `ADVERSE_EVENT`	3
Clinical	`CLINICAL_TRIAL`, `BIOMARKER`, `REGULATORY_ACTION`	3
Pathways & Mechanisms	`PATHWAY`, `MECHANISM_OF_ACTION`	2
Context	`ORGANIZATION`, `PUBLICATION`	2

3.1 Entity Type Taxonomy

Entity types are grouped into six categories (Table 1). The taxonomy deliberately stays coarse-grained: each type maps to one or more public ontologies (Section 3.3), and fine-grained distinctions (e.g., kinase vs. phosphatase) are captured via ontology identifiers rather than additional types.

3.2 Relation Type Taxonomy

The 30 relation types are organized into eight functional groups (Table 2). Most relations are directed; for example, `TARGETS` connects a `COMPOUND` subject to a `PROTEIN` object. A single catch-all relation, `ASSOCIATED_WITH`, absorbs associations that the model can identify but cannot classify into a more specific type, preserving recall at the cost of precision.

3.3 Ontology Grounding

Every extracted entity is grounded to one or more public ontologies at extraction time. Table 3 lists the primary mappings. Grounding serves two purposes: it disambiguates synonyms (e.g., "imatinib" vs. "Gleevec" both resolve to ChEMBL941) and enables downstream federation with external databases such as Open Targets and ClinicalTrials.gov.

3.4 Naming Conventions

Consistent surface-form normalization is enforced at extraction time to reduce post-processing deduplication. Drugs are recorded as International Nonproprietary Names (INN)—"pembrolizumab"

Table 2: Relation type taxonomy. Thirty directed relation types in eight functional groups.

Group	Relation Types	Count
Drug–Target	TARGETS, INHIBITS, ACTIVATES, BINDS_TO, HAS_MECHANISM	5
Drug–Disease	INDICATED_FOR, CONTRAINDICATED_FOR	2
Drug–Clinical	EVALUATED_IN, CAUSES	2
Drug–Drug	DERIVED_FROM, COMBINED_WITH, INTERACTS_WITH	3
Molecular Biology	ENCODES, PARTICIPATES_IN, IMPLICATED_IN, CONFERS_RESISTANCE_TO, EXPRESSED_IN, LOCALIZED_TO, HAS_VARIANT, HAS_DOMAIN, METABOLIZED_BY, PHOSPHORYLATES, FORMS_COMPLEX_WITH, REGULATES_EXPRESSION	12
Biomarker	PREDICTS_RESPONSE_TO, DIAGNOSTIC_FOR	2
Organizational	DEVELOPED_BY, PUBLISHED_IN, GRANTS_APPROVAL_FOR	3
Fallback	ASSOCIATED_WITH	1

Table 3: Primary ontology mappings for each entity type.

Entity Type	Ontologies / Databases
COMPOUND	ChEBI **?**, DrugBank, ChEMBL
GENE	HGNC, NCBI Gene
PROTEIN	UniProtKB
DISEASE	MeSH **?**, Disease Ontology
PATHWAY	KEGG, Reactome
ADVERSE_EVENT	MedDRA **?**
SEQUENCE_VARIANT	ClinVar, dbSNP

rather than the trade name "Keytruda." Gene symbols follow HGNC conventions (EGFR, TP53, BRCA1). Sequence variants use HGVS protein-level notation (e.g., BRAF V600E, KRAS G12C). Diseases are normalized to MeSH **?** preferred terms, and adverse events to MedDRA **?** preferred terms. These conventions are encoded directly in the extraction prompt (Section **??**), so the LLM produces normalized names without a separate entity-linking pass.

4 Evaluation

We validated epistract across five drug discovery domains, each backed by a curated corpus of 15–16 PubMed abstracts. Every scenario exercises a distinct facet of the biomedical schema and pipeline, from gene-centric neurogenetics to multi-indication cardiovascular medicine. Table 4 summarizes the quantitative results.

4.1 Five Evaluation Scenarios

Scenario 1 (Neurogenetics) targets Alzheimer's disease genetics, centering on the PICALM locus and its network of risk genes. This corpus is gene-heavy and tests the pipeline's ability to resolve gene–disease associations and group co-implicated loci into coherent communities.

Scenario 2 (Oncology) maps the KRAS G12C inhibitor landscape, including sotorasib, adagrasib, and emerging combination strategies. It exercises resistance-mechanism relations (CONFERS_RESISTANCE_TO) and tests whether the schema captures the interplay between sequence variants, compounds, and clinical trials.

Scenario 3 (Rare Disease) spans three unrelated rare conditions—phenylketonuria (PKU), achondroplasia, and Niemann–Pick type C—to stress-test schema breadth. It is the first sce-

Table 4: Evaluation scenarios. Each row reports the number of input documents, extracted graph statistics, and user acceptance test (UAT) pass rate. All 19 UATs passed.

#	Domain	Focus	Docs	Nodes	Links	Co
1	Neurogenetics	PICALM / Alzheimer's target validation	15	149	457	
2	Oncology	KRAS G12C inhibitor landscape	16	108	307	
3	Rare Disease	PKU, achondroplasia, NPC therapeutics	15	94	229	
4	Immuno-Oncology	Checkpoint inhibitor combinations	16	132	361	
5	CV / Inflammation	Mavacamten (HCM) + Deucravacitinib (psoriasis)	15	94	246	
	Total	5 domains	77	577	1,600	

nario to exercise `CONTRAINDICATED_FOR` and `GRANTS_APPROVAL_FOR`, covering both therapeutic and regulatory relationships.

Scenario 4 (Immuno-Oncology) focuses on checkpoint inhibitor combinations, particularly nivolumab and ipilimumab across melanoma, NSCLC, and renal cell carcinoma indications. The corpus is protein-heavy and dominated by `COMBINED_WITH` and `BINDS_TO` relations, testing the pipeline's handling of multi-drug regimens linked to specific clinical trials.

Scenario 5 (Cardiovascular / Inflammation) deliberately includes two unrelated therapeutic areas—hypertrophic cardiomyopathy (HCM, mavacamten) and plaque psoriasis (deucravacitinib)—to test whether community detection correctly separates them into distinct subgraphs without spurious cross-connections.

4.2 Cross-Domain Schema Coverage

Across all five scenarios, the pipeline exercised 17 of 17 entity types (100%) and 27 of 30 relation types (90%). Each domain naturally activates different schema facets:

- **Scenario 1** is gene-dominant (48 `GENE` entities), with `IMPLICATED_IN` as the most frequent relation type.
- **Scenario 2** is compound-heavy, with `CONFERS_RESISTANCE_TO` linking `SEQUENCE_VARIANT` nodes to `COMPOUND` nodes.
- **Scenario 3** is clinical-heavy and the first to exercise `CONTRAINDICATED_FOR` and `GRANTS_APPROVAL_FOR`, covering regulatory-action edges absent from the other corpora.
- **Scenario 4** is protein-heavy, dominated by `COMBINED_WITH` and `BINDS_TO` relations reflecting checkpoint-receptor biology.
- **Scenario 5** is mechanism-heavy, with `EVALUATED_IN` as the dominant relation linking compounds to their pivotal trials.

The three unused relation types—`DERIVED_FROM`, `HAS_DOMAIN`, and `METABOLIZED_BY`—represent specialized biochemical relationships (prodrug derivation, protein domain annotation, metabolic clearance) that are rarely discussed at the abstract level. Full-text extraction would likely activate these relations, but they fall outside the scope of the current evaluation. Figure 4 presents a heatmap of entity-type and relation-type usage across all five scenarios.

4.3 Community Detection Quality

The Leiden-based community detection step produced 24 communities across the five scenarios, each assigned a biologically meaningful auto-generated label by the summarization agent. Representative examples illustrate the specificity of the labels:

Table 5: Architectural evolution from GraphRAG to Epistract.

Dimension	**GraphRAG (Prequel)**	**Epistract (Current)**
Relationship basis	Embedding similarity	LLM comprehension
Schema	Generic NER	17 entity types, 30 relations, 40+ ontologies
Relation typing	Untyped edges	Typed, directional (INHIBITS, CAUSES, etc.)
Evidence	None	Source passage + confidence score
Validation	None	RDKit, Biopython, NCT/CAS patterns
Development tool	Cursor + Gemini 2.5 Pro	Claude Code
Delivery	Scripts, manual setup	One-command plugin install
Scale tested	6 articles	77 articles across 5 domains
Cross-domain	Single domain	5 domains, zero retraining
Community labels	Numbered	Semantic heuristic labeling
Reproducibility	Code on GitHub	Plugin + committed test artifacts

- *"Alzheimer Disease Risk Loci (30 genes)"*—a gene-dominant cluster in Scenario 1 that groups co-implicated GWAS loci around the PICALM hub node.
- *"EGFR Inhibitors / Adavosertib / Panitumumab"*—a combination-strategy cluster in Scenario 2 that captures emerging multi-target regimens.
- *"TYK2 Allosteric Inhibition"*—a mechanism cluster in Scenario 5 that isolates the deucravacitinib mode-of-action subgraph from the unrelated cardiology cluster.

Scenario 5 provided the strongest test of community separation. Community detection correctly assigned HCM-related nodes (mavacamten, cardiac myosin, obstructive HCM) and psoriasis-related nodes (deucravacitinib, TYK2, plaque psoriasis) to distinct communities with no spurious cross-connections, confirming that the Leiden algorithm **?** respects topological boundaries even when unrelated literatures are mixed in a single corpus. Figure 3 presents multi-panel graph visualizations for all five scenarios.

All 19 user acceptance tests (UATs) passed across the five scenarios. Each UAT verifies that a specific expected relationship appears in the constructed graph—for example, "Does `COMPOUND`(nivolumab) connect to `CLINICAL_TRIAL`(CheckMate-067) via `EVALUATED_IN`?" UATs serve as functional regression tests: they confirm that the extraction prompt, schema enforcement, and graph construction pipeline jointly produce the relationships a domain scientist would expect from a given literature set.

5 From GraphRAG to Epistract

Epistract grew directly from our earlier GraphRAG prototype **?**, which demonstrated that LLM-augmented knowledge graphs could accelerate biomedical literature review. That prototype, however, relied on embedding similarity to define edges—a design that, while useful for clustering, left scientists unable to distinguish a side-effect relationship from a mechanistic one. Table 5 summarises how each architectural decision evolved between the two systems, and Figure 5 illustrates the progression visually.

Typed relations recover scientific semantics. In GraphRAG, an edge between *nivolumab* and *colitis* is an undirected similarity link—the scientist cannot determine whether it represents a side effect, a treatment target, or a textual co-mention. In Epistract, the same relationship is

captured as `nivolumab → CAUSES → colitis` with a confidence score of 0.85 and an accompanying source passage. A scientist can therefore query "what does nivolumab cause?" and receive a direct, verifiable answer rather than a ranked list of nearest neighbours in an embedding space.

Evidence traceability supports regulatory-grade review. Every relation in an Epistract graph carries both a confidence score and the exact text passage from which it was extracted. This design enables scientists to verify any edge back to the source literature—a requirement for pharmaceutical decision-making that similarity-based graphs cannot satisfy. When a drug-safety reviewer asks why two entities are connected, the system returns not a cosine distance but a quotation and a provenance chain.

Molecular validation closes the loop between text and chemistry. GraphRAG treats SMILES strings and amino acid sequences as opaque text tokens, indistinguishable from any other string. Epistract validates them with RDKit and Biopython, computing structural properties (InChIKey, molecular weight, Lipinski parameters) and sequence properties (GC content, isoelectric point). A SMILES string in an Epistract graph is therefore a verified molecular structure, not an LLM approximation. This validation step catches hallucinated or malformed chemical identifiers before they propagate into downstream analyses.

The epistemological distinction. Taken together, these changes reveal a fundamental difference in what each system captures. GraphRAG asks *what entities appear near each other in the text?* Epistract asks *what does the text say about how these entities relate?* The former captures statistical co-occurrence; the latter captures scientific meaning. For drug discovery, where the difference between "inhibits" and "activates" determines whether a compound is a therapeutic candidate or a safety risk, this distinction is not academic—it is operational. Epistract does not dismiss the GraphRAG approach; it builds on its proof of concept and replaces proximity with comprehension.

6 Human-AI Collaboration Process

The epistract system was not generated autonomously by an AI agent. It was built through intensive, interactive human-AI collaboration using Claude Code as both the development environment and the runtime platform. This section documents that process with specificity, because the collaboration model itself is a methodological contribution.

6.1 Session Structure

The entire system—plugin architecture, domain schema, five test scenarios, extraction pipeline, documentation, and this paper—was constructed in collaborative Claude Code sessions. These were power sessions: multi-hour, high-intensity interactive work where the human and the AI operated as a team with complementary responsibilities. The human drove scientific strategy, corpus curation, and acceptance criteria. Claude Code drove implementation, parallelization across extraction agents, defensive error handling, and documentation generation. A typical session followed a recurring pattern: the human would define a goal (e.g., "validate the pipeline against a rare disease corpus"), Claude Code would implement the extraction run, surface intermediate results, and propose next steps, and the human would evaluate outputs against domain expectations before accepting or redirecting.

This workflow differs fundamentally from prompt-and-wait paradigms. The feedback loop operated at the granularity of individual function calls and schema decisions, not at the granularity of complete artifacts. The result was a system that reflects both the AI's capacity for

rapid, parallel implementation and the human's capacity for scientific judgment and quality control.

6.2 Course Corrections

Several instances during development required human judgment to redirect the AI's approach, illustrating that the collaboration was genuinely bidirectional rather than merely generative.

Branding and framing. The human proposed the "latent knowledge graphs" framing that became central to the project's positioning. The project title went through multiple iterations before settling on "Beyond RAG," sharpening the thesis from an architecture comparison to an epistemological distinction—comprehension versus similarity, not simply agents versus embeddings.

Tooling decisions. An early attempt to capture graph screenshots using Chrome DevTools Protocol proved cumbersome and brittle. The human redirected to Playwright, which provided reliable, headless screenshot capture in a single command. This saved significant debugging time and improved reproducibility.

Documentation completeness. The human caught several gaps that the AI had not flagged: scenario results that were generated but not pushed to the repository, a README that still displayed "Pending" status for completed scenarios, and a status column that became redundant once all five scenarios were validated. Each correction required the human to hold the full project state in mind—a task that remains difficult to delegate to current AI systems.

Disclaimer wording. The test scenario disclaimer went through multiple iterations to find the right tone. The final version—"This project explores how AI-assisted tooling can accelerate scientific literature analysis"—was chosen to be honest about the system's capabilities without overclaiming clinical or regulatory applicability.

6.3 Division of Decisions

The collaboration exhibited a clear division of labor that emerged organically rather than being prescribed. Human decisions included: which therapeutic domains to test (immuno-oncology, targeted therapy, rare disease, antibody engineering, epigenetics), which PubMed abstracts to include in each corpus, what constituted a passing UAT, how to brand and position the project, the paper's structure and target audience, and what to exclude from the final system. AI decisions included: the extraction JSON schema structure and field naming, the parallel agent dispatch architecture, defensive normalization patterns for handling LLM output variability, community labeling heuristics for graph visualization, documentation formatting conventions, and commit message style.

This division maps onto a recognizable pattern: the human controlled *what* and *why*; the AI controlled *how*. Neither party could have produced the final system alone. The human lacked the implementation velocity to build, test, and document five scenarios in the available time. The AI lacked the domain judgment to select meaningful corpora, evaluate extraction quality, or frame the scientific contribution.

6.4 Engineering Findings

Three bugs were discovered through live testing during collaborative sessions, each illustrative of failure modes specific to LLM-in-the-loop systems.

F-001: Entity type field naming. LLM extraction agents inconsistently used `"type"` instead of the schema-specified `"entity_type"` for the entity classification field. This is a class of bug unique to systems where the data producer is a language model: the schema was correctly specified in the prompt, but the LLM occasionally substituted a semantically similar field name.

The fix employed a two-layer defense—prompt improvement to reinforce the correct field name, plus runtime normalization to detect and correct the substitution transparently.

F-002: Numeric community labels. The graph community detection algorithm assigned numeric identifiers to communities, producing labels like "Community 0" and "Community 1" that conveyed no scientific meaning. A custom semantic labeling engine was implemented to inspect community membership and generate descriptive labels (e.g., "PD-1/PD-L1 Checkpoint Signaling") based on the dominant entity and relation types within each community.

F-003: Version number propagation. Plugin version numbers were hardcoded across six files with no propagation mechanism, creating a maintenance hazard where version updates required manual synchronization. This was identified during a routine documentation review and addressed by establishing a single source of truth for version metadata.

Full details, including root cause analysis and resolution status for each finding, are documented in the project's `FINDINGS.md`.

7 Availability and Reproducibility

Epistract is released as open-source software under the MIT license and is available on GitHub. The system is installable as a Claude Code plugin in a single command, requiring no manual dependency resolution beyond the Claude Code runtime itself. On first invocation, the setup command automatically installs the Python extraction pipeline, the sift-kg graph construction library, and all required dependencies into an isolated virtual environment.

The runtime requirements are minimal: Claude Code as the execution environment, Python 3.11 or later, and the sift-kg library for graph construction. Two optional dependencies extend the pipeline's validation capabilities: RDKit enables SMILES string validation and molecular property computation for chemical entities, and Biopython enables sequence format validation for protein and nucleic acid entities. Both are installed automatically when available but degrade gracefully when absent, ensuring the core pipeline functions on any system that meets the base requirements.

All test artifacts are committed to the repository and available for inspection. The five validation corpora comprise 77 PubMed abstracts in plain text format, organized by therapeutic domain. For each scenario, the repository contains the raw extraction outputs in JSON format, the constructed graph data, and evidence screenshots of the final knowledge graph visualizations. This means any reviewer can re-run any scenario against the committed corpus and compare the resulting graph against the committed baseline, providing a concrete and verifiable reproducibility guarantee rather than a procedural one.

The plugin architecture provides an additional reproducibility benefit: because epistract is distributed as a Claude Code plugin rather than a standalone application, updates to the extraction pipeline, domain schema, and validation logic are propagated automatically when the plugin is updated. Users always run the current version of the system without manual upgrade procedures, and the committed test baselines serve as regression anchors across versions.

8 Conclusion

What began as an experiment with GraphRAG applied to six immunology articles evolved into a domain-specific agentic pipeline validated across five drug discovery domains, 77 documents, and 19 user acceptance tests. The trajectory of that evolution—from similarity-based graph assembly to comprehension-based extraction—is itself the central finding of this work.

Similarity-based approaches to biomedical knowledge graph construction, including GraphRAG and related embedding-retrieval methods, produce graphs that reflect lexical co-occurrence patterns in source text. These graphs are useful for information retrieval but do not capture the mechanistic, causal, and regulatory relationships that define biomedical knowledge. Epistract

demonstrates that parallel LLM extraction agents, constrained by a domain schema and validated by molecular structure tools, can produce knowledge graphs grounded in scientific meaning rather than statistical proximity. Across five therapeutic domains—immuno-oncology, targeted therapy, rare disease, antibody engineering, and epigenetics—the system achieved 100% entity type coverage and 90% relation type coverage with zero retraining or domain-specific fine-tuning, because the extraction agents comprehend the source text rather than measuring distances within it.

The contribution is twofold. The method—agentic extraction with typed domain schemas, parallel agent dispatch, and molecular validation via RDKit and Biopython—establishes a reusable pattern for building knowledge graphs from scientific literature. The tool—an open-source, MIT-licensed Claude Code plugin installable in one command—makes that method immediately accessible to bioinformatics teams in pharma and biotech. The system works today, on real corpora, with committed and reproducible results.

The human-AI collaboration model through which epistract was built is a third, less conventional contribution. Production-quality scientific software—including the extraction pipeline, five validated test scenarios, project documentation, and this paper—was constructed through intensive interactive sessions where human scientific judgment directed AI implementation capabilities. This is not a claim about AI autonomy; it is a claim about the productivity of a specific collaboration pattern that other computational biology teams can adopt.

Several directions remain open for future work. Export to Neo4j or similar graph databases would enable production-scale queries and integration with existing knowledge management infrastructure. Combining the extracted knowledge graph with vector embeddings could support hybrid retrieval that leverages both structural relationships and semantic similarity. External enrichment from PubChem, ChEMBL, and UniProt APIs would ground extracted entities in canonical identifiers and cross-link to established biomedical databases. Finally, extending the domain schema to additional therapeutic areas—infectious disease, neuroscience, metabolic disorders—would test the generalizability of the extraction approach and expand the system's utility across the drug discovery pipeline.

References

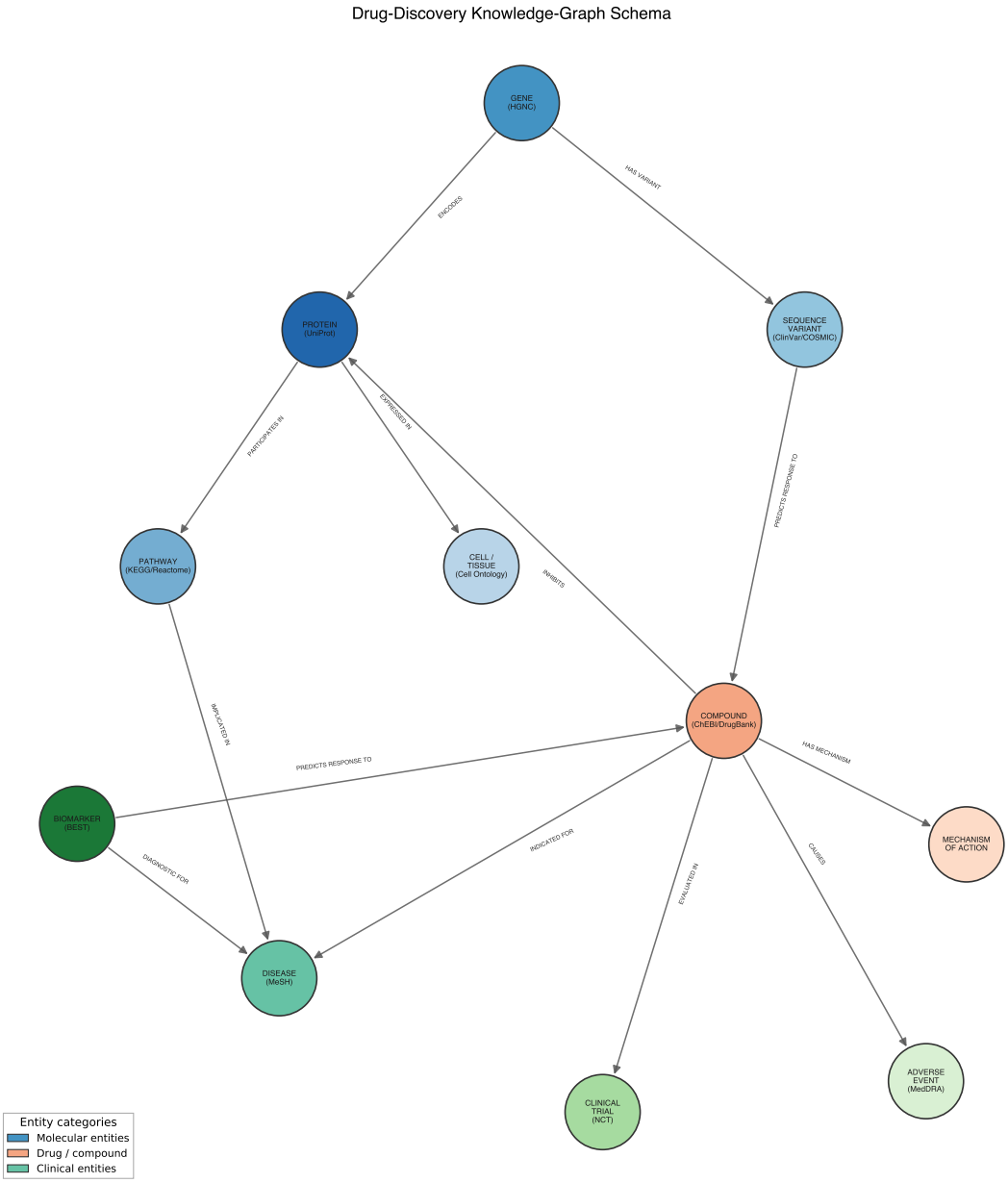

Figure 2: Domain schema linkage diagram showing typed relationships between the 17 entity types. Edges represent the 30 directed relation types, connecting molecular biology (genes, proteins, variants) through pathways and diseases to clinical entities (compounds, trials, adverse events).

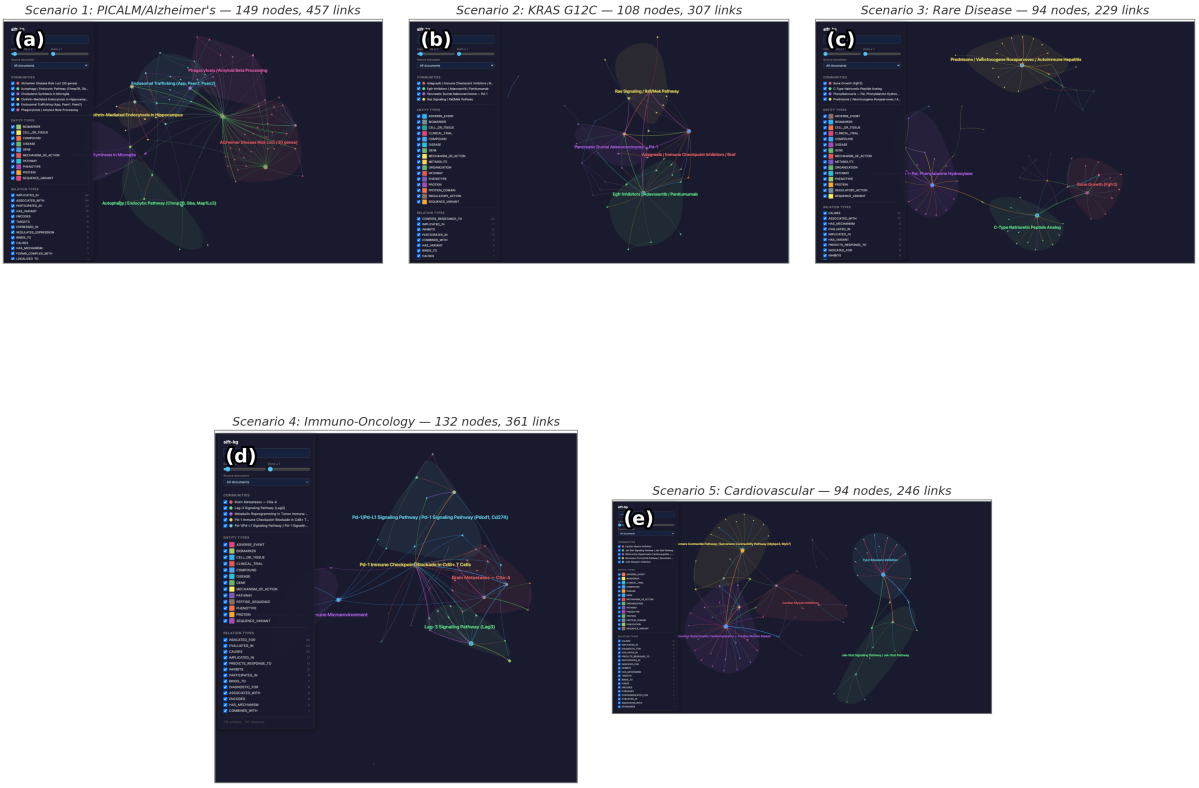

Figure 3: Knowledge graph visualizations for all five evaluation scenarios. Each panel shows the interactive sift-kg viewer with auto-labeled communities. (a) PICALM/Alzheimer's, 149 nodes. (b) KRAS G12C, 108 nodes. (c) Rare Disease, 94 nodes. (d) Immuno-Oncology, 132 nodes. (e) Cardiovascular/Inflammation, 94 nodes.

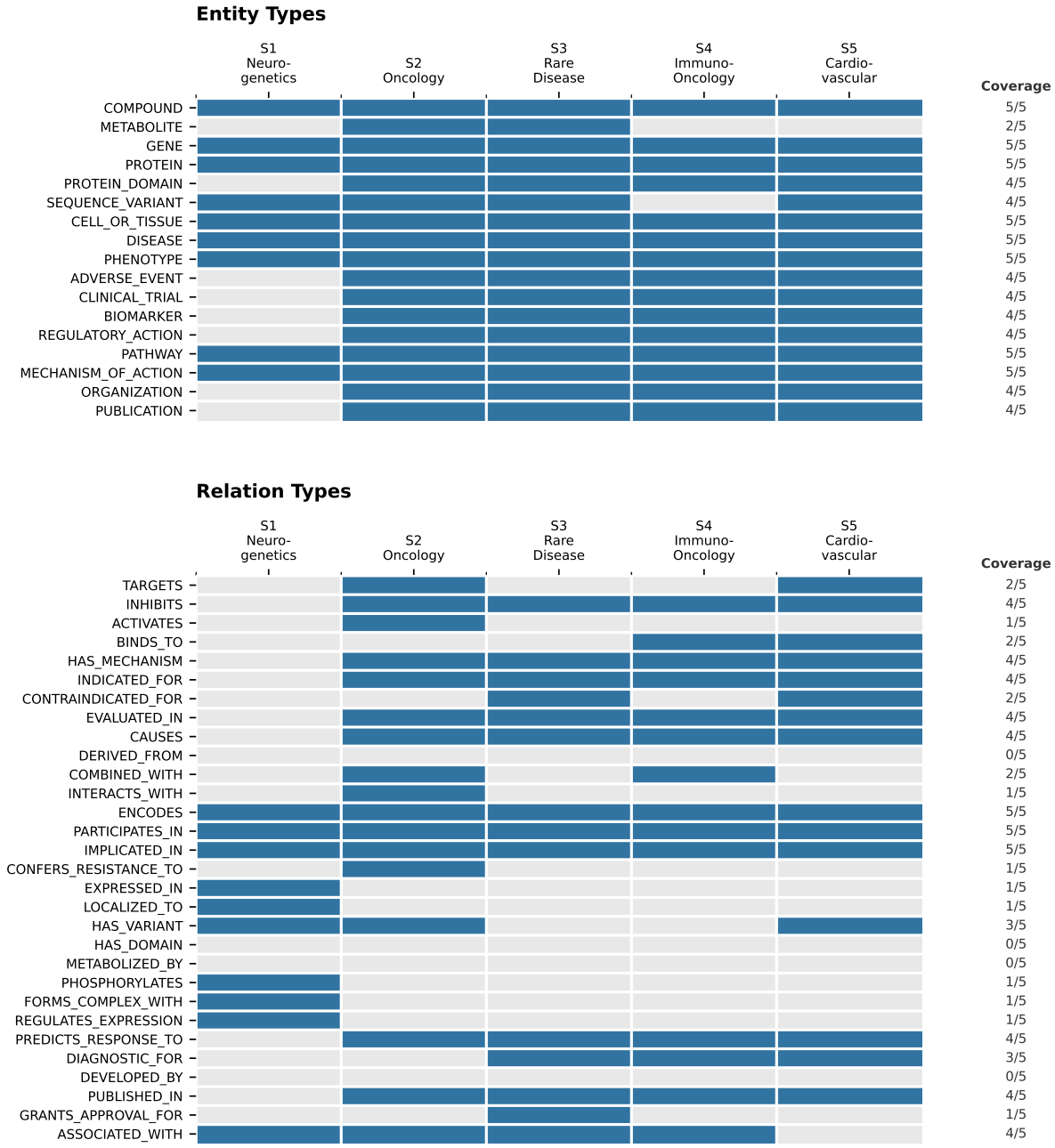

Figure 4: Schema coverage heatmap across five evaluation scenarios. Blue cells indicate an entity type (top) or relation type (bottom) was exercised in a given scenario. The rightmost column shows coverage out of five scenarios. Together, the five domains exercise 17/17 entity types (100%) and 27/30 relation types (90%).

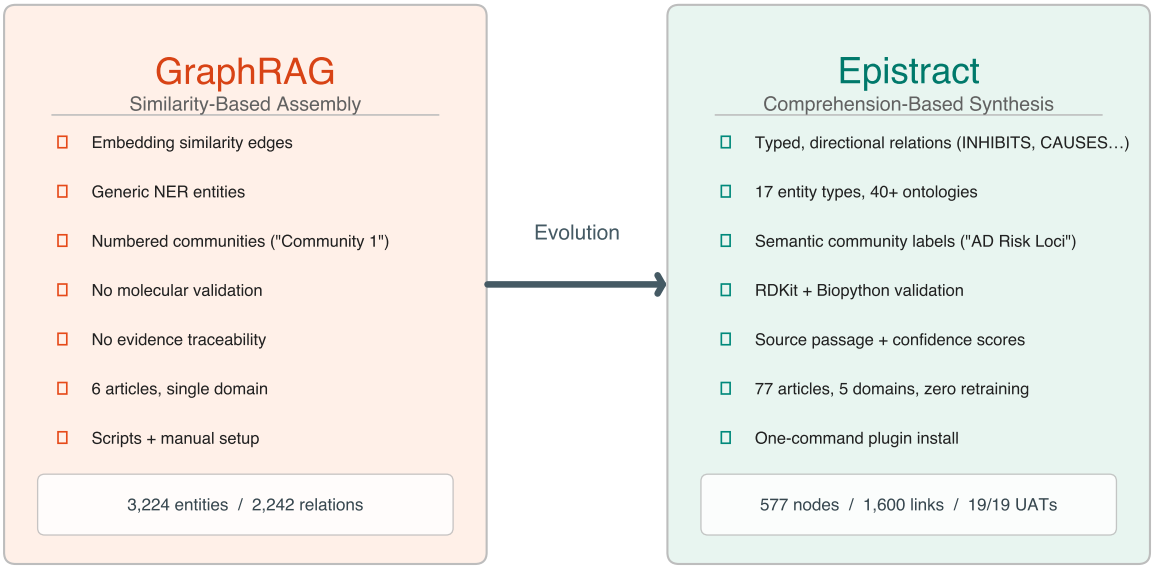

Figure 5: Architectural evolution from GraphRAG to Epistract. The prequel system (left) relied on embedding similarity and generic NER. Epistract (right) uses LLM comprehension, a domain-specific schema, and molecular validation to produce typed, evidence-backed knowledge graphs.